

# Tipo `void*`

---

`void*`

Questi lucidi sono basati sui lucidi di Programmazione 2 del Prof. Damiani e della Prof.ssa Baroglio



# Il Tipo `void*`

# Motivazione: Riutilizzo del Codice

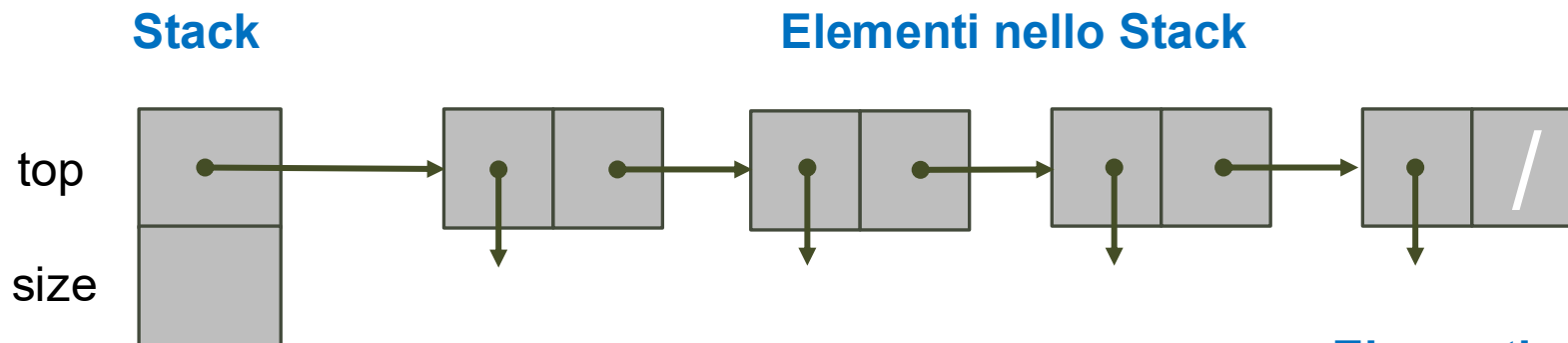
- Consideriamo questa situazione:
  - Dobbiamo gestire una pila di valori di tipo string (`char *`) e una pila di valori di tipo `ContactPtr`
    - Un ADT implementato con una versione opaca del tipo contatto di `contact.h` – cf. ESERCIZIO 2
  - Avendo già a disposizione l'ADT `charStackADT.h / charLinkedListStackADT.c`
  - Per farlo, potremmo scrivere:
    - Un ADT `stringStackADT.h / stringLinkedListStackADT.c`
    - E un ADT `contactOpqStackADT.h / contactOpqLinkedListStackADT.c`
  - Il cui codice differisce da `charStackADT.h / charLinkedListStackADT.c` solo per il tipo:
    - `char *` o `ContactPtr` invece di `char`
  - È mai possibile che non esista un modo per scrivere un'ADT pila usabile per elementi di qualsiasi tipo?

# Il Tipo `void *`



- Un valore di tipo `void *` contiene l'indirizzo di memoria per un tipo sconosciuto.
  - Lo spazio di memoria occupato dal puntatore `void *` è noto al compilatore
    - È quello occupato da un puntatore di qualsiasi altro tipo
  - Lo spazio di memoria occupato dal valore referenziato dal puntatore non è noto al compilatore
    - Il compilatore deve conoscere il tipo del valore a cui un puntatore fa riferimento per determinare il numero di byte che rappresentano il valore di riferimento.
- Quindi:
  - Non è possibile dereferenziare un puntatore di tipo `void *`
    - Dereferenziare un puntatore `void *` è un errore di sintassi.
  - Nota:
    - A tutte le variabili di tipo puntatore è possibile assegnare un valore di tipo `void *`, e viceversa.

# Esempio: Pila di `void*` con Liste



- Pila generica
  - Stesso codice di `CharStackADT.h`
  - Tranne `void*` al posto di `char`
- Attenzione:
  - La pila memorizza adesso puntatori
    - Validità dei puntatori è responsabilità del codice cliente

## Elementi nello Stack, i.e., Lista

```
typedef struct listNode ListNode,
    *ListNodePtr;

struct listNode {
    void* data;
    ListNodePtr next;
};
```

## Stack

```
struct stack {
    ListNodePtr top;
    int size;
};
```

# Pila di `void*` con Liste: `push()` e `pop()`

- Aggiunge un elemento
  - In testa alla lista

```
_Bool push(StackADT s, void *e) {  
    // Crea nodo lista  
    ListNodePtr ptr = malloc(sizeof(*ptr));  
    // malloc fallita?  
    if (ptr == NULL)  
        return 0;  
    // Salva dato  
    ptr->data = e;  
    // Inserisci in testa  
    ptr->next = s->top;  
    s->top = ptr;  
    // Incrementa dimensione stack  
    s->size++;  
    return 1;  
}
```

- Toglie e restituisce l'elemento in cima alla pila

```
void *pop(StackADT s) {  
    // Lista vuota?  
    if (isEmptyStack(s)) {  
        return NULL;  
    }  
    // Salvo dato in testa  
    void *e = s->top->data;  
    // Cancella nodo di testa  
    ListNodePtr ptr = s->top;  
    s->top = ptr->next;  
    free(ptr);  
    // Decrementa dimensione stack  
    s->size--;  
    // Restituisce dato  
    return e;  
}
```

# Pila di void\*: Esempio d'Uso



```
#include <stdio.h>
#include "stackADT.h"

int main(void) {
    int a[] = {2, 4, 8};

    printf("Stampa in ordine:\n");
    for(int i = 0; i < sizeof(a)/sizeof(*a); i++) {
        printf(" %d\n", a[i]);
    }

    // Reverse print a using a stack
    StackADT s = mkStack();
    for(int i = 0; i < sizeof(a)/sizeof(*a); i++) {
        push(s, &a[i]); // Aggiungo gli indirizzi
    }
    printf("La pila contiene %d elementi\n", stackSize(s));

    printf("Stampa in ordine inverso:\n");
    for(int i = 0; i < sizeof(a)/sizeof(*a); i++) {
        printf(" %d\n", *(int*)pop(s)); // Converto void* in int* per deferenziarlo
    }
    printf("La pila è vuota? %d\n", isEmptyStack(s));
}
```